

Parameter Sniffing

#sql-server #performance #query-plan #optimizer

What It Is

When SQL Server compiles a stored procedure, it sniffs the parameter values from the first execution and builds a query plan optimized for those specific values. That plan is then cached and reused for all subsequent executions, regardless of what values future callers pass.

This is usually a good thing. The optimizer uses the sniffed values to estimate cardinality (row counts), choose join strategies, and decide between seeks and scans. For most workloads, the first-caller plan works well enough for everyone.

The problem arises when data distribution is skewed. If the first caller passes an atypical value (say, a tote with 500 shipments when most totes have 3), the cached plan is optimized for 500 rows: hash joins, large memory grants, scans. Every subsequent caller with a 3-row tote pays the cost of that heavy plan.

How to Detect It

The telltale sign is plan instability in **Query Store Triage**: the same query_id appears with multiple plan_id values, and their performance metrics vary by 10x to 100x. Every time the plan gets evicted and recompiled (memory pressure, schema change, statistics update), a new first-caller value produces a different plan shape.

```
-- Find queries with multiple plans and high read variance
Select
    qsq.query_id,
    Count(Distinct qsp.plan_id) As PlanCount,
    Min(qsrs.avg_logical_io_reads) As MinAvgReads,
    Max(qsrs.avg_logical_io_reads) As MaxAvgReads,
    Max(qsrs.avg_logical_io_reads) / NullIf(Min(qsrs.avg_logical_io_reads), 0) As ReadRatio
From sys.query_store_query qsq
Join sys.query_store_plan qsp On qsp.query_id = qsq.query_id
Join sys.query_store_runtime_stats qsrs On qsrs.plan_id = qsp.plan_id
Group By qsq.query_id
Having Count(Distinct qsp.plan_id) > 1
Order By Max(qsrs.avg_logical_io_reads) Desc
```

The Local Variable Mitigation

The classic fix is to assign input parameters to local variables and use only the locals in your queries:

```
-- Parameters as declared
@StationId VarChar(8),
@FillType VarChar(1),
@ToteId VarChar(24),
@BatchFillId Int

-- Local copies. The optimizer cannot sniff these.
Declare @LocalStationId VarChar(8)
Declare @LocalFillType VarChar(1)
Declare @LocalToteId VarChar(24)
Declare @LocalBatchFillId Int
```

```
Set @LocalStationId = @StationId
Set @LocalFillType = @FillType
Set @LocalToteId = @ToteId
Set @LocalBatchFillId = @BatchFillId
```

Why it works: the optimizer knows it can't see the runtime value of a local variable at compile time, so it falls back to density-based estimates. It uses the **Density Vector** statistics (average rows per distinct value) instead of the histogram for a specific value. This produces a middle-of-the-road plan that is never optimal for any single value but is acceptably good for all values.

When NOT to Use It

- No skew in the data. If every distinct value has roughly the same number of rows, the sniffed plan works fine for everyone. Check with **Density Vector** analysis first.
- Parameterless procs. No parameters means nothing to sniff. The issue is elsewhere.
- OPTION (RECOMPILE) is viable. If the proc runs infrequently (under 100 times per day), forcing a fresh compile each time gives the optimizer perfect information every time. For high-frequency procs (thousands of calls per day), recompile overhead is unacceptable.

The Tradeoff

Local variables give you plan stability at the cost of plan optimality. You will never get the best possible plan for a specific value, but you will also never get a catastrophically bad plan. For high-frequency OLTP procedures, stability almost always wins.

Real-World Example

In `isp_RxfGetListOfManualFillGroups` (v48 to v49), we added local variable copies for all four input parameters. The proc's Query Store data showed 6 different plans with a 100x read range (24K to 2.4M reads). After the local variable mitigation, the optimizer consistently picks a density-based plan that lands around 20K reads. That isn't the theoretical minimum, but it is reliably fast.

Related Concepts

- **Density Vector**: what the optimizer uses when it can't sniff.
- **Query Store Triage**: how to detect plan instability.
- **Table Variables vs Temp Tables**: another statistics-related issue.
- **Non-SARGable Predicates**: another way to accidentally prevent good estimates.